IT运维系统

结构：SSM框架+JSP

hlnms-webapp：controller层+视图层

- controller controller 层 --命名： XXXController
- webapp 视图层
  - app 页面
  - resource js脚本 css样式文件 图片等资源
  - WEB-INF/tags 自定义标签

hlnms-service：service层

- api 接口 --命名：XXXServiceAPI
- impl 实现类 --命名：XXXService

hlnms-dao: dao层

- api 接口 --命名：XXXDaoAPI
- resources/mapper 对应Sql xml文件 -命名：-XXXMapper

关系：

- XXXService implements XXXServiceAPI service实现service接口
- XXXService 调用 XXXDaoAPI 进行数据库操作
- XXXDaoAPI 与 -XXXMapper 一一对应 在Mapper中编写Sql语句
- XXXController 控制层调用 XXXService service层实现相关业务

hlnms-probe （重点）

根据不同协议采集各个资源指标

数据表

- md_motype 模型类型表

字段 | 索引 | 外键 | 触发器 | 选项 | 注释 | SQL 预览

名	类型	长度	小数点	不是 null	键	注释
MOTYPEID	decimal	10	0	☑	1	模型的ID
MOTYPECODE	varchar	128	0	☑		模型的ID
MOTYPENAME	varchar	128	0	☑		模型的名称
ISTOP	varchar	1	0	☑		0 : TOP 1 : SUB
MOTYPETABLE	varchar	70	0	☑		实例表名 OBJ_MOTYPEID
CREATEUSER	varchar	64	0	☑		模型的ID
CREATETIME	datetime	0	0	☑		模型的ID
UPDATEUSER	varchar	64	0	☐		模型的ID
UPDATETIME	datetime	0	0	☐		模型的ID

-

ISTOP:0表示该模型为顶层对象，1表示子对象

可以根据MOTYPETABLE字段获取该模型下所有资源所在的表 **obj_xxxx**

- **obj_xxxx 资源表 xxx为对应模型类别ID**

对象 | model_jg @s... | model_jf @s... | model_jg_jf_r... | model_jg_obj... | model_obj_p... | md_motype ... | md_motype ... | md_motype_... | md_test_indic... | obj_10001 @... | cfg_vendor

开始事务 | 文本 | 筛选 | 排序 | 导入 | 导出

OBJID	OBJNAME	OBJCODE	IP	OBJINDEX	MOTYPEID	INSERTTIME	ISCOLLECTION	ALARMSTATUS	LASTCONFTIME	LASTPERFTIME
4087479	后台管理服务器	192.168.0.167	192.168.0.167	(Null)	10001	2019-06-01 13:32:45	1	(Null)	2021-12-17 00:00:20	(Null)
4087833	数据库服务器	192.168.0.3	192.168.0.3	(Null)	10001	2019-07-24 10:03:53	1	(Null)	2021-08-28 00:00:20	(Null)
4088087	监控系统服务器	监控系统服务器	192.168.0.169	(Null)	10001	2019-08-14 16:16:12	1	(Null)	2021-12-17 00:00:21	(Null)
4088160	halu03	192.168.0.3_SNMP	192.168.0.3	(Null)	10001	2021-08-20 14:52:05	0	(Null)	2021-08-28 00:00:20	(Null)
4088758	192.168.0.101	192.168.0.101	192.168.0.101	(Null)	10001	2021-10-16 02:14:56	1	(Null)	2021-12-17 00:00:20	(Null)
4088759	192.168.0.102	192.168.0.102	192.168.0.102	(Null)	10001	2021-10-16 02:15:25	1	(Null)	2021-12-17 00:00:21	(Null)
4088760	192.168.0.103	192.168.0.103	192.168.0.103	(Null)	10001	2021-10-16 02:15:55	1	(Null)	2021-12-17 00:00:21	(Null)
4088761	192.168.0.104	192.168.0.104	192.168.0.104	(Null)	10001	2021-10-16 02:16:25	1	(Null)	2021-12-17 00:00:21	(Null)
4088762	192.168.0.106	192.168.0.106	192.168.0.106	(Null)	10001	2021-10-16 02:16:51	1	(Null)	2021-12-17 00:00:21	(Null)
4088763	192.168.0.107	192.168.0.107	192.168.0.107	(Null)	10001	2021-10-16 02:17:23	1	(Null)	2021-12-17 00:00:21	(Null)
4088764	192.168.0.108	192.168.0.108	192.168.0.108	(Null)	10001	2021-10-16 02:18:18	1	(Null)	2021-12-17 00:00:21	(Null)
4088765	192.168.0.109	192.168.0.109	192.168.0.109	(Null)	10001	2021-10-16 02:18:41	1	(Null)	2021-12-17 00:00:21	(Null)
4088766	192.168.0.167	192.168.0.167	192.168.0.167	(Null)	10001	2021-10-16 02:19:19	1	(Null)	2021-12-17 00:00:20	(Null)
4088767	192.168.0.168	192.168.0.168	192.168.0.168	(Null)	10001	2021-10-16 02:19:43	1	(Null)	(Null)	(Null)
4088768	192.168.0.169	192.168.0.169	192.168.0.169	(Null)	10001	2021-10-16 02:20:11	1	(Null)	2021-12-17 00:00:21	(Null)
4088769	192.168.0.241	192.168.0.241	192.168.0.241	(Null)	10001	2021-10-16 02:20:41	1	(Null)	2021-12-17 00:00:21	(Null)
4088770	192.168.0.182	192.168.0.182	192.168.0.182	(Null)	10001	2021-10-16 02:21:07	1	(Null)	(Null)	(Null)
4088771	192.168.0.183	192.168.0.183	192.168.0.183	(Null)	10001	2021-10-16 02:21:29	0	(Null)	2021-12-02 00:00:24	(Null)
4088772	192.168.0.243	192.168.0.243	192.168.0.243	(Null)	10001	2021-10-16 02:21:54	1	(Null)	2021-12-17 00:00:21	(Null)

上图为模型ID为10001（Linux服务器）下的部分资源

- **rlt_motype_obj 模型-资源关联表**

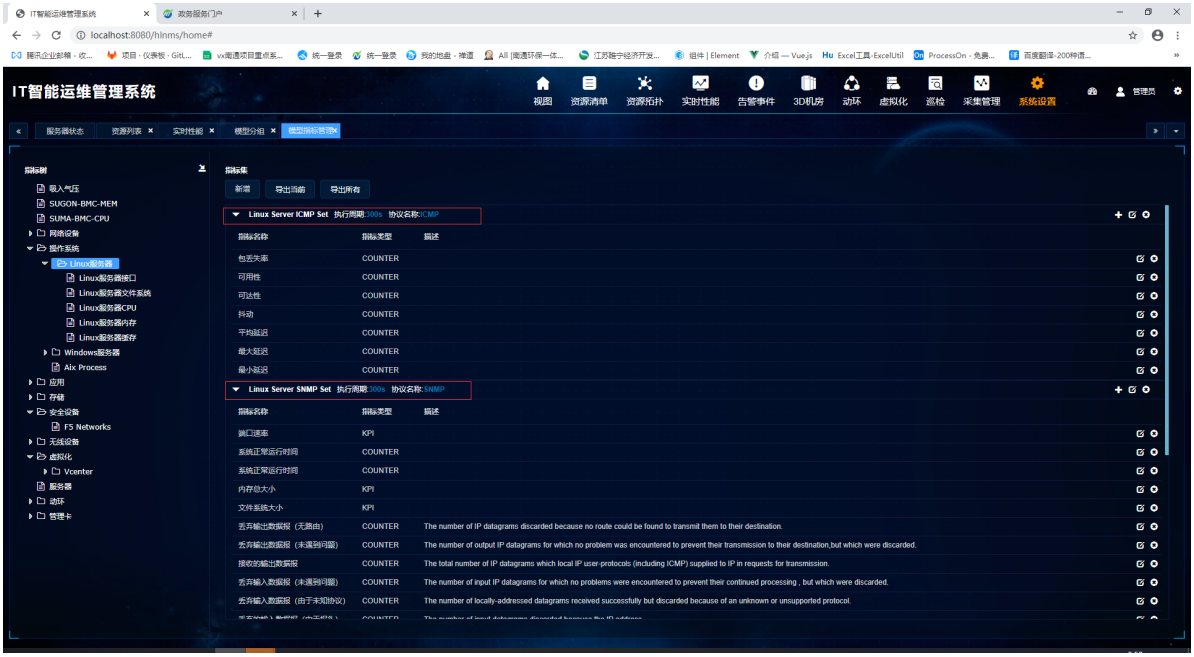
对象 | ... | md_indicat... | md_indicat... | md_indicat... | raw_10023... | pms_role_... | pms_role_... | pms_role ... | pms_role_... | pms_role_... | pms_role_...

开始事务 | 文本 | 筛选 | 排序 | 导入 | 导出

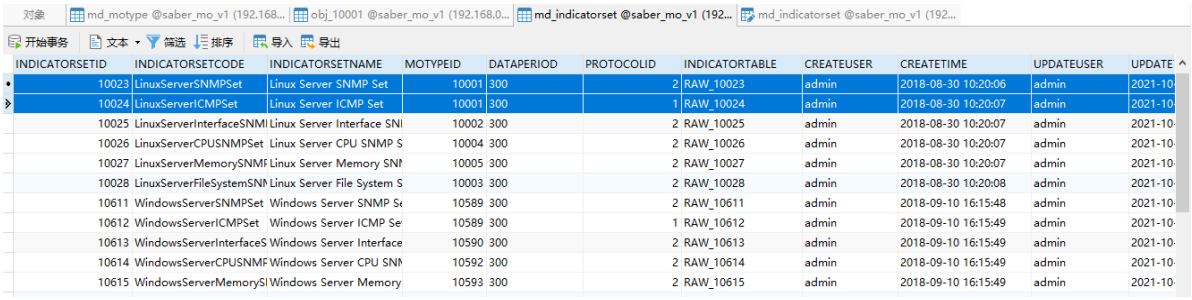
OBJID	MOTYPEID	PARENTID	PARENTMOTYPEID	TEMPLATEID	SERVERID
4088922	10004	4088760	10001	(Null)	(Null)
4088923	10004	4088760	10001	(Null)	(Null)
4088924	10004	4088760	10001	(Null)	(Null)
4088925	10004	4088760	10001	(Null)	(Null)
4088926	10004	4088760	10001	(Null)	(Null)
4088927	10003	4088768	10001	(Null)	(Null)
4088928	10003	4088768	10001	(Null)	(Null)
4088929	10003	4088768	10001	(Null)	(Null)
4088930	10003	4088768	10001	(Null)	(Null)
4088931	10003	4088768	10001	(Null)	(Null)
4088932	10003	4088768	10001	(Null)	(Null)
4088933	10003	4088768	10001	(Null)	(Null)
4088934	10003	4088766	10001	(Null)	(Null)
4088935	10003	4088766	10001	(Null)	(Null)
4088936	10003	4088766	10001	(Null)	(Null)
4088937	10003	4088766	10001	(Null)	(Null)
4088938	10003	4088766	10001	(Null)	(Null)
4088939	10003	4088766	10001	(Null)	(Null)
4088940	10003	4088766	10001	(Null)	(Null)
4088941	10003	4088758	10001	(Null)	(Null)
4088942	10003	4088758	10001	(Null)	(Null)
4088943	10003	4088758	10001	(Null)	(Null)
4088944	10003	4088758	10001	(Null)	(Null)
4088945	10003	4088758	10001	(Null)	(Null)
4088946	10003	4088758	10001	(Null)	(Null)
4088947	10003	4088758	10001	(Null)	(Null)

- **资源ID**
- **资源类型**
- **父资源ID**
- **父资源类型**

每一个模型有其对应的指标集



以Linux服务器为例 该模型有两个指标集 ICMP指标集和SNMP指标集 对应的数据表为**md_indicatorset**

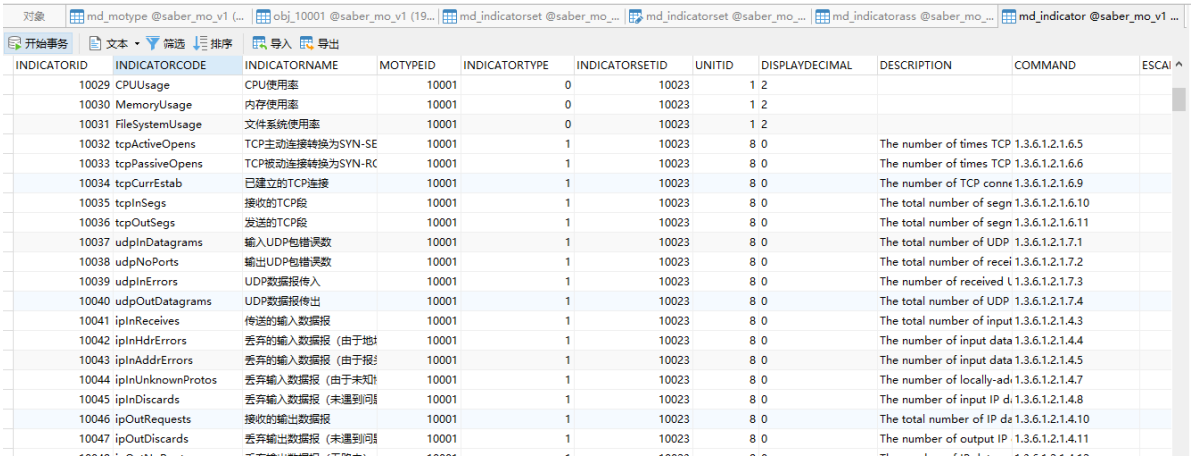


INDICATORSETID	INDICATORSETCODE	INDICATORSETNAME	MOTYPEID	DATAPERIOD	PROTOCOLID	INDICATORTABLE	CREATEUSER	CREATETIME	UPDATEUSER	UPDATE
10023	LinuxServerSNMPSet	Linux Server SNMP Set	10001	300	2	RAW_10023	admin	2018-08-30 10:20:06	admin	2021-10
10024	LinuxServerICMPSet	Linux Server ICMP Set	10001	300	1	RAW_10024	admin	2018-08-30 10:20:07	admin	2021-10
10025	LinuxServerInterfaceSNMI	Linux Server Interface SNI	10002	300	2	RAW_10025	admin	2018-08-30 10:20:07	admin	2021-10
10026	LinuxServerCPUSNMPSet	Linux Server CPU SNMP S	10004	300	2	RAW_10026	admin	2018-08-30 10:20:07	admin	2021-10
10027	LinuxServerMemorySNMI	Linux Server Memory SNI	10005	300	2	RAW_10027	admin	2018-08-30 10:20:07	admin	2021-10
10028	LinuxServerFileSystemSNI	Linux Server File System S	10003	300	2	RAW_10028	admin	2018-08-30 10:20:08	admin	2021-10
10611	WindowsServerSNMPSet	Windows Server SNMP Se	10589	300	2	RAW_10611	admin	2018-09-10 16:15:48	admin	2021-10
10612	WindowsServerICMPSet	Windows Server ICMP Se	10589	300	1	RAW_10612	admin	2018-09-10 16:15:49	admin	2021-10
10613	WindowsServerInterfaceS	Windows Server Interface	10590	300	2	RAW_10613	admin	2018-09-10 16:15:49	admin	2021-10
10614	WindowsServerCPUSNMI	Windows Server CPU SNI	10592	300	2	RAW_10614	admin	2018-09-10 16:15:49	admin	2021-10
10615	WindowsServerMemorySI	Windows Server Memory	10593	300	2	RAW_10615	admin	2018-09-10 16:15:49	admin	2021-10

关键字段：

- INDICATORSETID：指标集ID
- INDICATORSETCODE：指标集编码
- INDICATORSETNAME：指标集名称
- **MOTYPEID：所属模型ID**
- **INDICATORTABLE：指标数据所对应的表**

指标所在的表为**md_indicator**



INDICATORID	INDICATORCODE	INDICATORNAME	MOTYPEID	INDICATORTYPE	INDICATORSETID	UNITID	DISPLAYDECIMAL	DESCRIPTION	COMMAND	ESCAI
10029	CPUUsage	CPU使用率	10001	0	10023	1	2			
10030	MemoryUsage	内存使用率	10001	0	10023	1	2			
10031	FileSystemUsage	文件系统使用率	10001	0	10023	1	2			
10032	tcpActiveOpens	TCP主动连接转换为SYN-SE	10001	1	10023	8	0	The number of times TCP	1.3.6.1.2.1.6.5	
10033	tcpPassiveOpens	TCP被动连接转换为SYN-RC	10001	1	10023	8	0	The number of times TCP	1.3.6.1.2.1.6.6	
10034	tcpCurrEstab	已建立的TCP连接	10001	1	10023	8	0	The number of TCP conne	1.3.6.1.2.1.6.9	
10035	tcpInSegs	接收的TCP段	10001	1	10023	8	0	The total number of segn	1.3.6.1.2.1.6.10	
10036	tcpOutSegs	发送的TCP段	10001	1	10023	8	0	The total number of segn	1.3.6.1.2.1.6.11	
10037	udpInDatagrams	输入UDP包错误数	10001	1	10023	8	0	The total number of UDP	1.3.6.1.2.1.7.1	
10038	udpNoPorts	输出UDP包错误数	10001	1	10023	8	0	The total number of recei	1.3.6.1.2.1.7.2	
10039	udpInErrors	UDP数据报传入	10001	1	10023	8	0	The number of received U	1.3.6.1.2.1.7.3	
10040	udpOutDatagrams	UDP数据报传出	10001	1	10023	8	0	The total number of UDP	1.3.6.1.2.1.7.4	
10041	ipInReceives	传送的输入数据报	10001	1	10023	8	0	The total number of input	1.3.6.1.2.1.4.3	
10042	ipInHdrErrors	丢弃的输入数据报（由于地	10001	1	10023	8	0	The number of input data	1.3.6.1.2.1.4.4	
10043	ipInAddrErrors	丢弃的输入数据报（由于报	10001	1	10023	8	0	The number of input data	1.3.6.1.2.1.4.5	
10044	ipInUnknownProtos	丢弃输入数据报（由于未知	10001	1	10023	8	0	The number of locally-ad	1.3.6.1.2.1.4.7	
10045	ipInDiscards	丢弃输入数据报（未遇到问	10001	1	10023	8	0	The number of input IP d	1.3.6.1.2.1.4.8	
10046	ipOutRequests	接收的输出数据报	10001	1	10023	8	0	The total number of IP da	1.3.6.1.2.1.4.10	
10047	ipOutDiscards	丢弃输出数据报（未遇到问	10001	1	10023	8	0	The number of output IP	1.3.6.1.2.1.4.11	

字段：

名	类型	长度	小数点	不是 null	键	注释
INDICATORID	decimal	10	0	☑	1	指标ID
INDICATORCODE	varchar	128	0	☑		指标编码
INDICATORNAME	varchar	128	0	☑		指标名称
MOTYPEID	decimal	10	0	☑		模型ID
INDICATORTYPE	decimal	1	0	☑		0:KPI 1:COUNTER
INDICATORSETID	decimal	10	0	☑		指标集ID
UNITID	decimal	10	0	☑		单位ID
DISPLAYDECIMAL	varchar	1	0	☑		保留小数点位数
DESCRIPTION	varchar	256	0	☐		描述
COMMAND	varchar	4000	0	☐		采集指标命令
ESCAPE	varchar	256	0	☐		结果转义
RULEID	decimal	10	0	☐		规则ID
CALCULATION	varchar	512	0	☐		计算方式
COLUMNNAME	varchar	10	0	☑		指标对应列名
CREATETIME	datetime	0	0	☑		创建时间
CREATEUSER	varchar	64	0	☑		创建用户
UPDATETIME	datetime	0	0	☐		更新时间
UPDATEUSER	varchar	64	0	☐		更新用户
ISVIEW	varchar	1	0	☐		0:hide 1:view

关键字段：

- INDICATORID：指标ID
- MOTYPEID：模型ID
- INDICATORTYPE：指标集ID
- COMMAND：采集指标命令（非KPI指标）
- CALCULATION：计算方式（如果是KPI指标，此处存储的是计算规则）
- COLUMNNAME：对应列名（KPI__XX,即该指标在指标采集表中所在的列）

以指标10029（CPU使用率为例）

- 该指标为模型10001（Linux服务器）
- 10023（LinuxServerSNMPSet）指标集中的指标
- 是一个KPI指标 计算公式为 AVG{#[Linux服务器CPU].[Linux Server CPU SNMP Set].[cpu使用率]}
- 在数据表中对应的列为 KPI_1 由指标集可知对应的表为RAW_10023

综合来看就是 Linux服务器模型的 LinuxServerSNMPSet指标集中的 CPU使用率指标 存放在 RAW_10023表的 KPI_1 字段

角色管理：

数据表：

- pms_role　角色表
- pms_role_menu　角色-菜单关联表 **角色ID-菜单ID**
- pms_role_motype 角色-模型关联表 **角色ID-模型ID-对应资源数据表名**
- pms_role_node 角色-资源管理表　**角色ID-资源ID-模型ID**

资源列表功能模块

查询实体列表：getEntityItems（） 查询设备级的对象数据，包含统计子对象数量、过滤条件、分页

参数：

- pageSize：分页大小
- curPage：当前页面
- alarmId：告警状态ID（默认-1） 关联**rlt_obj_alarmstatus**表
- venderId：销售公司ID（默认-1）关联**cfg_vendor**表
- domainId：所属域ID（默认-1）关联**cfg_domain**表
- motypeId：模型ID（默认-1）
- searching：搜索内容（默认“”）

1、PageBounds绑定分页

```
PageBounds pageBounds = new PageBounds(curPage, pageSize);
```

2、根据当前用户获取该用户对应的资源结点,**查找用户资源权限，返回设备级对象ID**

```
List<String> objIds =
commonServiceAPI.getResCodeList(Common.getRemoteUser(request));
```

- 2.1如果当前用户为管理员（userType=1），则返回NULL

```
pmsDaoAPI.findUserType(userId) == 1
```

- 2.2 否则根据用户查找结点列表　如果为空插入负值为过滤参数
- ```
  List<String> list = pmsDaoAPI.user_hasNodes(userId);
  if (list.size() == 0)
    {
       list.add("-888888");
    }
  ```

- user_hasNodes 根据用户ID 获取其有权限的资源ID
  - 1、根据用户ID查询用户角色，根据**pms_role_node**表的关联查找具体资源
  - 2、根据用户ID查询用户角色，根据**pms_role_motype**表找到相应的模型ID,再根据**rlt_motype_obj**找到模型下的资源
  - 3、两者取并集

3、若objIds为null或空，说明该用户无有权限的资源，直接返回null，否则根据资源ID查询具体资源信息
  ```

3.1、设置数据源

```
DataSourceContextHolder.setTargetDataSource(DataSourceType.model);
```

3.2、查询所有模型信息,获取对应模型所存的数据表表名

```
List<Map<String, Object>> params = resourceCount.getObjectsMoTables();
```

3.3、若模型不为空，则查询资源信息，参数：分页、用户权限下的资源ID列表、模型列表、搜索内容、告警状态、销售商ID、模型ID、区域ID

```
moinfos = resourceCount.getAllEntitesForCount(pageBounds, objIds, params,
searching, alarmId, venderId,
        motypeId, domainId);
```

查询所有资源信息： 遍历模型列表，从模型中获取资源所在数据表名 拼接到sql语句中

```
<foreach collection="params" item="item" index="key" separator="union">
        SELECT
        O.OBJID,O.OBJNAME,O.OBJCODE,O.IP,O.MOTYPEID,case when O.VENDORID is null
then 0 else O.VENDORID end as
        VENDORID,O.MODEL,V.VENDORNAME,O.DOMAIN AS DOMAINID,D.DOMAINNAME,
        '${item.MOTYPETABLE}' as MOTYPETABLE,
        '${item.MOTYPENAME}' as MOTYPENAME,
        '${item.MOTYPECODE}' as MOTYPECODE
        FROM
        ${item.MOTYPETABLE} O LEFT JOIN cfg_vendor V ON O.VENDORID=V.VENDORID
LEFT JOIN cfg_domain D ON
        O.DOMAIN=D.DOMAINID
</foreach>
```

增加限制条件，查询满足筛选条件的数据：

```
<select id="getAllEntitesForCount" resultType="java.util.HashMap"
parameterType="java.util.Map" databaseId="mysql">
    SELECT
    A.*, CASE OA.ALARMSTATUS WHEN 0 THEN 7 ELSE OA.ALARMSTATUS END AS
    ALARMSTATUS,L.ALARMLEVELCOLOR,L.ALARMLEVELNAME,L.ALARMLEVELRGB
    FROM
    (
    <foreach collection="params" item="item" index="key" separator="union">
        SELECT
        O.OBJID,O.OBJNAME,O.OBJCODE,O.IP,O.MOTYPEID,case when O.VENDORID is null
then 0 else O.VENDORID end as
        VENDORID,O.MODEL,V.VENDORNAME,O.DOMAIN AS DOMAINID,D.DOMAINNAME,
        '${item.MOTYPETABLE}' as MOTYPETABLE,
        '${item.MOTYPENAME}' as MOTYPENAME,
        '${item.MOTYPECODE}' as MOTYPECODE
        FROM
        ${item.MOTYPETABLE} O LEFT JOIN cfg_vendor V ON O.VENDORID=V.VENDORID
LEFT JOIN cfg_domain D ON
        O.DOMAIN=D.DOMAINID
    </foreach>
```

```
    ) A
    LEFT JOIN rlt_obj_alarmstatus OA ON A.OBJID = OA.OBJID LEFT JOIN
cfg_alarmlevel L ON
    OA.ALARMSTATUS=L.ALARMLEVELID
    <where>
        <if test="objIds!=null">
            A.OBJID IN
            <foreach item="item" index="index" collection="objIds" open="("
separator="," close=")">
                #{item}
            </foreach>
        </if>
        <if test="searching!=null">
            <bind name="pattern" value="'%' + searching + '%'"/>
            AND (UPPER(OBJNAME) LIKE '${pattern}' OR IP LIKE '${pattern}')
        </if>
        <if test="alarmId!=-1">
            AND ALARMSTATUS=#{alarmId}
        </if>
        <if test="venderId!=-1">
            AND VENDORID=#{venderId}
        </if>
        <if test="motypeId!=-1">
            AND MOTYPEID=#{motypeId}
        </if>
        <if test="domainId==0">
            AND (A.DOMAINID IS NULL OR A.DOMAINID='')
        </if>
        <if test="domainId!=-1 and domainId!=0">
            AND A.DOMAINID=#{domainId}
        </if>
    </where>
    ORDER BY
    ALARMSTATUS ASC,OBJID DESC
</select>
```

4、根据资源ID，查询该资源下子资源的数量

```
 for (Map<String, Object> info : moinfos)
            {
                long objId = Long.parseLong(info.get("OBJID").toString());
                List<Map<String, Object>> subCounts =
resourceCount.getSubTypeCountOfEntity(objId);
                info.put("subCounts", subCounts);
            }
```

根据**rlt_motype_obj** 表中父ID和父类型来分组计数

```
        SELECT
            COUNT(MO.MOTYPEID) AS NUM,
            MO.MOTYPEID,
            M.MOTYPENAME,
            M.MOTYPETABLE
        FROM rlt_motype_obj MO, md_motype M
        WHERE MO.PARENTID = #{objId} AND MO.MOTYPEID = M.MOTYPEID
        GROUP BY MO.MOTYPEID, M.MOTYPENAME, M.MOTYPETABLE
```

查询资源下所属子对象信息

getSubEntites（）

参数:

- moTable: 子资源所在的表（由查询资源列表时返回的subCounts中的MOTYPETABLE确定）
- objId: 资源ID（父资源）
- subMotypeId: 子模型ID
- searching: 搜索条件
- pageSize: 分页大小
- curPage: 当前页
- sortName:
- sortOrder:

新增模型

API: addMoType（）

参数;

- code(moTypeCode) 模型编码
- name(moTypeName) 模型名称
- isTop 0：顶层对象 1：子对象
- groupId：分组Id **md_group**
- parentIds：父对象Id
- id：0：新增
- attrs：属性(基础属性+自定义属性)
- commonMotypeId：公共类型 **md_motype_common**

1、增加模型数据以及相关obj_xxx表 addMoTypeModel

参数:

- moTypeCode
- MoTypeName
- isTop
- request

```
HashMap<String, Object> map = new HashMap<>();
            map.put("moTypeCode", moTypeCode);
            map.put("moTypeName", MoTypeName);
            map.put("isTop", isTop);
            map.put("createUser",
com.halu.saber.service.Common.getRemoteUser(request));
            map.put("createtime", new Date());
modelTypeDaoAPI.saveMoType(map);
```

在模型表md_moType中新增一条数据，插入时根据获取到的模型ID，生成相应表名**obj_${id}**

```
 <selectKey keyProperty="id" resultType="java.lang.Integer" order="BEFORE"
databaseId="oracle">
        select seq_obj_id.nextval id from dual
 </selectKey>
 <selectKey keyProperty="id" resultType="java.lang.Integer" order="BEFORE"
databaseId="mysql">
        select nextval('SEQ_OBJ_ID')
 </selectKey>

     insert into Md_Motype
(motypeid,motypecode,motypename,ISTOP,motypetable,createuser,createtime)
     values (#{id}, #{moTypeCode},#{moTypeName},#{isTop},'obj_${id}',#
{createUser},#{createtime})
```

根据模型编码获取模型ID，生成obj_xxx表 **createObjTable(MoTypeId, request);**

首先判断obj_xxx表是否存在，若不存在，则动态生成属性列：

```
if (exist == 0)
    {
        List<String> columns = new ArrayList<String>();

    // 添加动态属性
    for (int i = 2; i <= COLUMNS_COUNT; i++)
    {
        String att = ATTRIBUTE + i;
        columns.add(att);
    }
    tableParam.put("columns", columns);
```

参数：

```
// 表名前缀
private static final String OBJ_TABLE = "OBJ_";
// 属性列前缀
private static final String ATTRIBUTE = "Att_";
// 列数量
private static final int COLUMNS_COUNT = 50;
```

2、如果groupId不为空 保存模型与分组之间的联系 (即在 rlt_motype_group 表中新增数据)

```
HashMap<String, Object> map = new HashMap<>();
map.put("moTypeId", id);
map.put("groupId", groupId);

modelTypeDaoAPI.saveMotypeGroup(map);
```

3、如果该模型有父类型 parentIds不为空 增加两者之间的关联（addMoRelation(parentIds, moTypeCode, request);）

在**rlt_motype_motype**表中插入数据

字段：

- MOTYPEID 模型ID
- PARENTMOTYPEID 父模型ID

4、如果有公共模型 commonMotypeId不为空 增加两者之间的关联（modelTypeDaoAPI.saveMotypeCommon(map);）

在**rlt_commonmotype_motype**中插入数据

字段：

- MOTYPEID 模型ID
- COMMONMOTYPEID 公共模型ID

5、如果属性不为空 attrs不为空，为模型增加属性

attrs：

- 基础属性（根据公共模型确定?）
- 自定义属性

数据结构：{"attrName":"属性1","unit":"","comAttrName":"*"}

- attrName：属性名
- unit：单位 （为空表示为text）
- comAttrName：基础属性（自定义属性为*）

方法：addMoAttribute(attrs, id, request, attributeMap);

参数：

- attrs：前端传入的属性（JSON）
- id：模型ID
- request:请求
- attributeMap：在新增模型时为空

addMoAttribute(attrs, id, request, attributeMap)

- 对数据进行处理 放入HashMap中

```
HashMap<String, Object> map = new HashMap<>();
map.put("attributeName", attributeJson.get("attrName"));
map.put("moTypeId", moTypeId);
map.put("createUser", com.halu.saber.service.Common.getRemoteUser(request));
map.put("createtime", new Date());
// 判断属性的单位是否为空，如果为空则为text，数据库保存为null
if (!ObjectUtils.isEmpty(attributeJson.get("unit")))
{
    map.put("attributeUnitl", attributeJson.get("unit"));
}
```

- 获取当前模型属性列数组 （modelTypeDaoAPI.getUserATTNum(moTypeId)）

```
select replace(t.columnname,'ATT_','') as NUM  from md_moattribute t where
t.motypeid=#{_parameter}
```

- 在**md_moattribute**表插入数据

```
if (!ATTCount.contains((i + 1) + ""))
{
    map.put("columnName", "ATT_" + (i + 1));      //在obj_xxx 中该属性对应于那一列
    // 判断之前是否存在该属性，若存在，则保留原属性ID
    if (!ObjectUtils.isEmpty(attributeMap.get(map.get("attributeName"))))
    {
        map.put("id", attributeMap.get(map.get("attributeName")));
        modelTypeDaoAPI.insertMoAttribute(map);
    }
    else
    {
        modelTypeDaoAPI.saveMoAttribute(map);
    }
}
```

- 如果为基础属性，需要插入属性与公共属性的关联关系 **rlt_commonattribute_attribute**

字段:

- MOTYPEID 模型ID
- ATTRIBUTEID 属性ID
- COMMONATTRIBUTEID 公共属性ID

```
if (!"*".equals(attributeJson.get("comAttrName")))
{
    // 校验没有问题开始插入数据
    map.put("commonAttributeName", attributeJson.get("comAttrName"));
    Map<String, Object> commonAttributeMap =
modelTypeDaoAPI.getCommonAttributeMap(map);
    String commonAttributeId =
String.valueOf(commonAttributeMap.get("COMMONATTRIBUTEID"));
    // 判断属性是否存在
    String attributeID = modelTypeDaoAPI.getAttrIdByNameAndModelType(map);
    map.put("attributeId", attributeID);
    map.put("commonAttributeId", commonAttributeId);
    modelTypeDaoAPI.insertRltComattrAttr(map);
}
```

新增指标

```
addIndicator(IndicatorModel indicatorModel, HttpServletRequest request)
```

参数:

- indicatorid 指标id，为0时表示新增，否则为更新指标
- indicatorcode 指标编码
- indicatorsetid 指标集编码
- motypeid 模型ID
- indicatortype 指标类型 0:KPI指标 1:普通指标
- isView 是否显示

md_indicator

如果是公共指标 需要增加两者之间的联系 **rlt_commonindicator_indicator**

```
// 数据库变动时通知后台  根据探针类型给探针发送消息
commonServiceAPI.sendMessageToProbe(ProbeType.Performance,
ProbeCommandName.update);
```

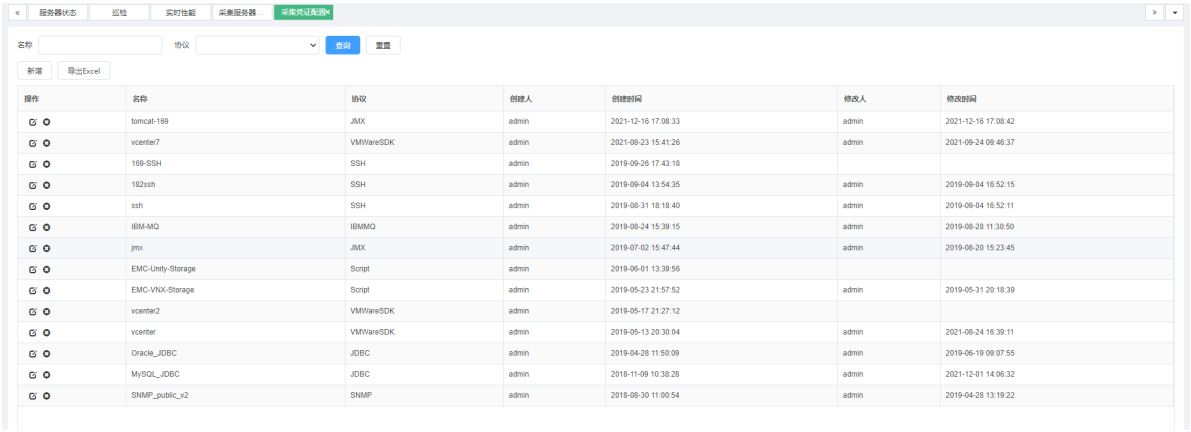
```
//枚举类
public enum ProbeType {
    Discover, Configuration, Performance, Dial, Event, Inspect, ConfigFile,
Process,vmware
}

public enum ProbeCommandName {
    start, stop, update, updateConfigs, status, realTime, realTimeConnect
}
```

sendMessageToProbe

1、根据探针类型 获取探针列表 **collect_probe**

```
List<String> probeIds = common.getProbeList(probeType.name());
```



操作	名称	协议	创建人	创建时间	修改人	修改时间
	tomcat-169	JMX	admin	2021-12-16 17:08:33	admin	2021-12-16 17:08:42
	vcenter7	VMWareSDK	admin	2021-08-23 15:41:26	admin	2021-09-24 09:46:37
	169-SSH	SSH	admin	2019-09-26 17:43:18		
	182ssh	SSH	admin	2019-09-04 13:54:35	admin	2019-09-04 16:52:15
	ssh	SSH	admin	2019-08-31 18:18:40	admin	2019-09-04 16:52:11
	IBM-MQ	IBMMQ	admin	2019-08-24 15:39:15	admin	2019-08-28 11:30:50
	jmx	JMX	admin	2019-07-02 15:47:44	admin	2019-08-20 15:23:45
	EMC-Unity-Storage	Script	admin	2019-06-01 13:39:56		
	EMC-VNX-Storage	Script	admin	2019-05-23 21:57:52	admin	2019-05-31 20:18:39
	vcenter2	VMWareSDK	admin	2019-05-17 21:27:12		
	vcenter	VMWareSDK	admin	2019-05-13 20:30:04	admin	2021-08-24 16:39:11
	Oracle_JDBC	JDBC	admin	2019-04-28 11:50:09	admin	2019-06-19 09:07:55
	MySQL_JDBC	JDBC	admin	2018-11-09 10:38:28	admin	2021-12-01 14:06:32
	SNMP_public_v2	SNMP	admin	2018-08-30 11:00:54	admin	2019-04-28 13:19:22

collect_credential 采集凭证表

CREDENTIALID	CREDENTIALNAME	PROTOCOLID	PARAM	CREATEUSER	CREATETIME
10001	SNMP_public_v2	2	{"port":"161","snmpVersion":"v2c","community":"SABERAESA0B0C3B16FB614213F322B91E13DE92F"}	admin	2018-08-30 11:00:
10005	MySQL_JDBC	3	{"userName":"root","passWord":"SABERAESB30828C8D25E99BC85978B1BCDE1B96E","dataType":"mysql","port":"3306","sid	admin	2018-11-09 10:38:
10019	Oracle_JDBC	3	{"userName":"nms","passWord":"SABERAES3B08C11523310DBFF6D5A74D547FA908","dataType":"oracle","port":"1521","typ	admin	2019-04-28 11:50:
10020	vcenter	11	{"port":"80","userName":"administrator@vsphere.local","passWord":"SABERAES18F5BAB9C67F433E46D7DEF7E7C094BB"}	admin	2019-05-13 20:30:
10021	vcenter2	11	{"port":"80","userName":"root","passWord":"SABERAES18F5BAB9C67F433E46D7DEF7E7C094BB"}	admin	2019-05-17 21:27:
10023	EMC-VNX-Storage	19	{"userName":"sysadmin","passWord":"SABERAES925F6BE2D0F36D4575CECBE33D056BBA"}	admin	2019-05-23 21:57:
10024	EMC-Unity-Storage	19	{"userName":"admin","passWord":"SABERAES1F17285BFB5B70F801BC9CE28A0D341F"}	admin	2019-06-01 13:39:
10025	jmx	6	{"port":"8888","userName":"monitorRole","passWord":"SABERAES3B08C11523310DBFF6D5A74D547FA908"}	admin	2019-07-02 15:47:
10026	IBM-MQ	20	{"ip":"192.168.0.80","port":"1414","channel":"QUEUE_RECV","userId":"MUSR_MQADMIN","password":"","ccsid":"1381","trans	admin	2019-08-24 15:39:
10027	ssh	5	{"userName":"root","passWord":"SABERAESAEEC65D47DF4CE65F432F3F9F52EB86C","port":"22","needEn":"on","enCmd":"",	admin	2019-08-31 18:18:
10028	182ssh	5	{"userName":"root","passWord":"SABERAES3B08C11523310DBFF6D5A74D547FA908","port":"22","needEn":"on","enCmd":"",	admin	2019-09-04 13:54:
10029	169-SSH	5	{"userName":"root","passWord":"SABERAES3B08C11523310DBFF6D5A74D547FA908","port":"22","needEn":"on","enCmd":"",	admin	2019-09-26 17:43:
10030	vcenter7	11	{"port":"80","userName":"administrator@vsphere.local","passWord":"SABERAESD141A9370566747AA5AEF5FDC265CE4A"}	admin	2021-08-23 15:41:
10031	tomcat-169	6	{"port":"8089","userName":"","passWord":"SABERAESAAB8AEB193E95240D05943A85B037DD5249AA54DA4B01F7D1A5DEB	admin	2021-12-16 17:08:

param与新增时参数对应 **PROTOCOLID** 为协议ID

指标采集任务：

使用Quartz来作为解决任务调度的工具（相较于Timer 可以处理复杂逻辑调度和类似于每个星期一12：00处理任务）

结构：

- **Job接口**：这个方法定义了需要调度的方法，开发者在使用Quartz并定义调度任务时候，需要实现这个接口并且重写此方法。

```
public interface Job {
    void execute(JobExecutionContext var1) throws JobExecutionException;
}
```

在本项目中使用**JobImpl**来实现该接口

```
public class JobImpl implements Job
{
    private static final Logger logger = LoggerFactory.getLogger(JobImpl.class);

    @Override
    public void execute(JobExecutionContext context) throws
JobExecutionException
    {
        //需要进行调度执行的任务
        TaskManager.getInstance().executeTask();
    }
}
```

- **JobDetail** JobDetail表示一个具体的可执行的调度程序，Job是这个可执行调度程序所要执行的内容，另外JobDetail还包含了这个任务调度的方案和策略

```
JobDetail jobDetail = JobBuilder
                      .newJob(JobImpl.class)    //所要执行的内容 即job的实现类
                      .withIdentity("collectTask", Scheduler.DEFAULT_GROUP)  //分
组标识
                      .build();
```

- **Trigger** 触发器 主要是触发Job执行的时间触发规则
  - SimpleTrigger
  - CronTrigger

CronTrigger 支持类似日历的重复间隔，而不是单一的时间间隔。

```
 // 触发器
TriggerBuilder<Trigger> triggerBuilder = TriggerBuilder.newTrigger();
// 触发器名,触发器组
triggerBuilder.withIdentity("collectTaskTrigger", Scheduler.DEFAULT_GROUP);
triggerBuilder.startNow();
// 触发器时间设定
triggerBuilder.withSchedule(CronScheduleBuilder.cronSchedule(taskPeriodStr));
// 创建Trigger对象
CronTrigger trigger = (CronTrigger) triggerBuilder.build();
```

- 通过triggerBuilder.build 来获得一个触发器实例
- 可以通过triggerBuilder来设置触发器属性 如触发器组、触发器时间等
- triggerBuilder.withSchedule() 设置触发时间 传入的是CronScheduleBuilder.cronSchedule(taskPeriodStr)
- CronScheduleBuilder.cronSchedule(taskPeriodStr) taskPeriodStr为cron表达式 表示执行的时间间隔
    - ```
      ProbeTaskQuartz=0 0/5 * * * ?    //每5分钟执行一次
      ```
- **Scheduler**代表一个Quartz的独立运行容器，Trigger和JobDetail可以注册到Scheduler中，二者在Scheduler中有各自的组件、名称和组

```
scheduler.scheduleJob(jobDetail, trigger);
```

Scheduler可以将Trigger绑定到某一个JobDetail上，当Trigger被触发时，对应的Job就被执行，一个Job可以对应多个Trigger，一个Trigger只能对应一个Job，

```
scheduler.start();
```

启动执行

每过5分钟后就会执行**TaskManager.getInstance().executeTask()**方法

- TaskManager.getInstance() 获得一个TaskManager的单例对象
- .executeTask()执行该对象下的executeTask方法

执行采集任务 executeTask（） （snmp协议）

1、首先根据配置文件获取探测任务周期(默认为300秒 即5分钟)

```
 String taskPeriodStr = ProbeContextHolder.getStringValue("ProbeTaskPeriod");
long taskPeriod = 300;
if (!ObjectUtils.isEmpty(taskPeriodStr) && ObjectUtils.isNumber(taskPeriodStr))
{
    taskPeriod = Long.parseLong(taskPeriodStr);
}
```

ProbeContextHolder 根据不同的探针类型读取相应的配置文件 在探针启动时读取相应的配置文件
```

ProbeContextHolder.getStringValue("ProbeTaskPeriod"); 获取配置文件 **core.properties** 中 **ProbeTaskPeriod**对应的值

2、获取任务开始时间

```
long nowTime = System.currentTimeMillis();
Date startTime = new Date(nowTime - nowTime % (taskPeriod * 1000));  //格式化时间 去除偏差 精确到 00秒
```

3、获取需要进行采集的对象列表

采集对象的操作基本都通过**ObjectManager**类来进行操作

```
List<ObjectModel> objectModelList = objectManager.getObjectList(
         ProbeContextHolder.getStringValue(Constants.PROBE_PROTOCOL),
System.getProperty(Constants.PROBE_ID));
```

objectManager.getObjectList（String protocol, String probeId）

参数：

- protocol 探针采集协议此处值为（“SNMP”）
  - **Constants.PROBE_PROTOCO** 常量 ProbeProtocol
  - 即 ProbeContextHolder.getStringValue（“ProbeProtocol”）
  - ```
    ProbeProtocol=SNMP
    ```
- probeId 探针ID 值为（“Probe-Perf-SNMP”）
  - **Constants.PROBE_ID** 常量 **probe.id**
  - 即 System.getProperty(“probe.id”)

（1）根据 probeId 探针ID获取各个模型及其对应协议下的指标集以及属性

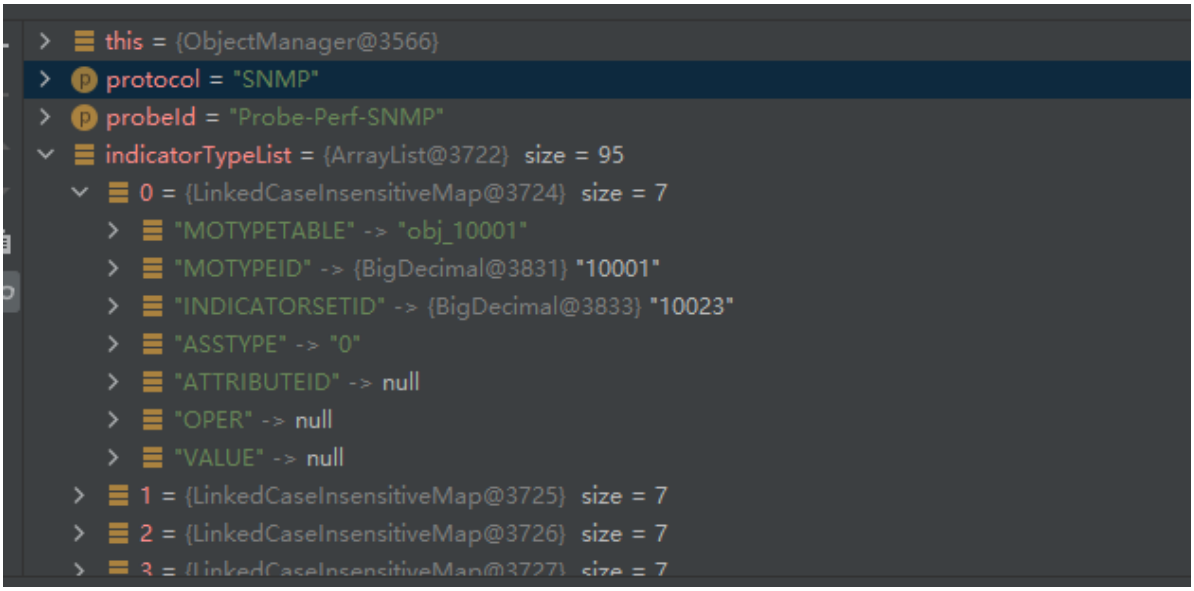
md_motype 根据 MOTYPEID关联 **md_indicatorset**

md_indicatorset 根据指标集ID关联 **md_indicatorass**

从**collect_probe**表和**cfg_protocol**表中获取对应的协议ID **protocolid**进行筛选

Probe-Perf-SNMP 对应的协议为 SNMP 对应的协议ID为 2

返回数据结构如下：
    ```



（2）遍历上方集合 如果 根据协议ID、探针ID、资源表名获取所有对象

```
allObjList = performanceDao.getObjectForAll(protocol, probeId, motypetable);
```

数据表：

- obj_xxx 资源实体表
- collect_credential 采集凭证表
- rlt_credentials_obj 采集凭证-资源联系表

限制条件：

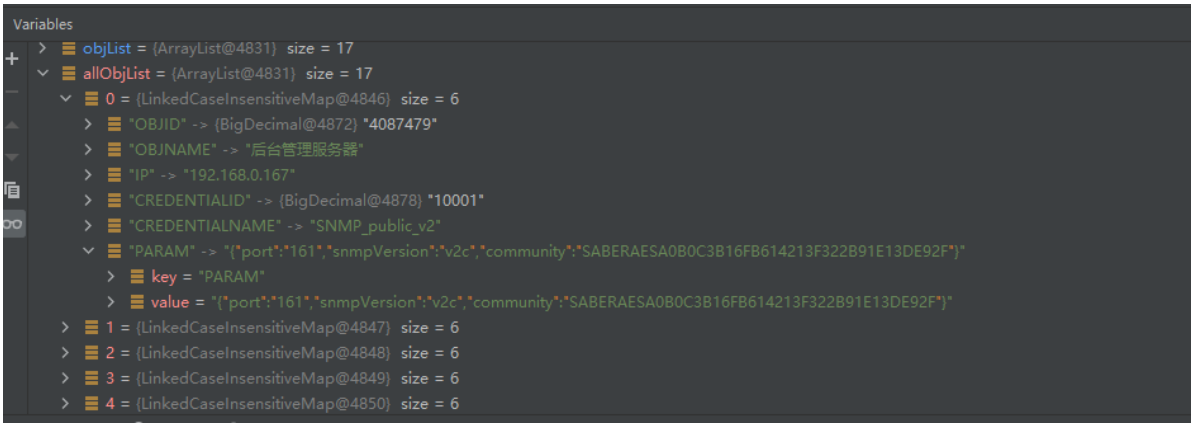
- OBJ.ISCOLLECTION = 1 资源是否采集标志位为 1
- 资源凭证的协议ID

```
(SELECT PROTOCOLID
FROM CFG_PROTOCOL
WHERE PROTOCOLNAME = ?)
AND OBJ.OBJID IN
( SELECT MO.OBJID FROM rlt_motype_obj MO , collect_probe P WHERE MO.SERVERID =
P.SERVERID AND P.PROBEID = ?)
```

SERVERID 作用暂不清楚

返回字段：

- 资源ID
- 资源名称
- 资源IP
- 凭证ID
- 凭证名称
- 凭证参数



根据asstype 对allObjList进行筛选 得到筛选后的数组objList

- 0：全部
- 1：动态
- 2：静态

遍历allObjList，以及凭证参数键值对，取出

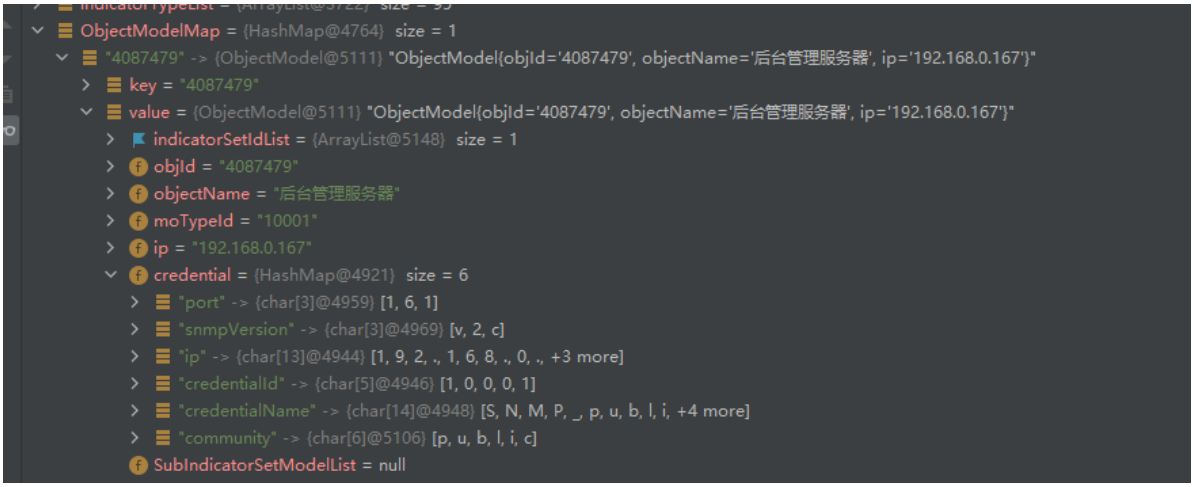
- credentialId 凭证ID
- credentialName 凭证名称
- ip 资源IP

凭证参数进行AES128接码后的键值对，以下为snmp凭证参数（不同协议参数不同）：

- port 端口
- snmpVersion snmp协议版本
- community 认证需要的community

将资源属性、采集指标集和上述的采集凭证信息封装到objectModel对象中：

```
if (ObjectUtils.isEmpty(ObjectModelMap.get(objid)))
                {
                    ObjectModel objectModel = new ObjectModel();
                    objectModel.setObjId(objid);
                    objectModel.setObjectName(objname);
                    objectModel.setIp(ip);
                    objectModel.setCredential(inMap);
                    objectModel.setMoTypeId(motypeId);
                    objectModel.setIndicatorSetIdList(new ArrayList<>());
                    if (objList.contains(objectMap))
                    {
                        objectModel.getIndicatorSetIdList().add(indicatorsetid);
                    }
                    ObjectModelMap.put(objid, objectModel);
                }
                else
                {
                    if (objList.contains(objectMap))
                    {
                        ObjectModel objectModel = ObjectModelMap.get(objid);
                        objectModel.getIndicatorSetIdList().add(indicatorsetid);
                        ObjectModelMap.put(objid, objectModel);
                    }
                }
```

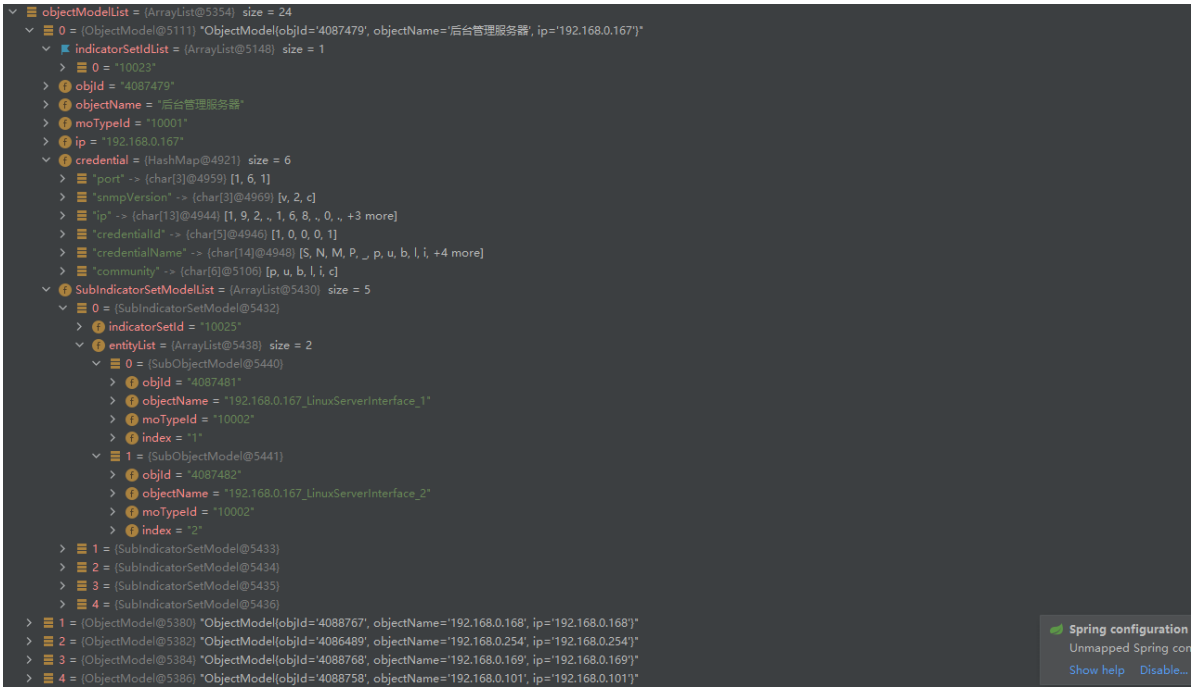


遍历objectModelList，为objModel加载所有的子对象指标集SubIndicatorSetModelList：

```
getObject(objectModel, objectModel.getMoTypeId(), objectModel.getObjId());
```

- 根据模型id moTypeId 获取所有子模型ID
- 根据子模型ID获取对象的指标集列表
- 遍历获取模型下的子资源
- 将处理后的数据存入SubIndicatorSetModelList列表

最终结果：



（4）获取到objectModelList后循环遍历

- 根据 objectModel.getIndicatorSetIdList() 中的指标集 获取相应指标模型
- 根据指标模型获取采集时间间隔，过滤掉不在时间间隔内的数据

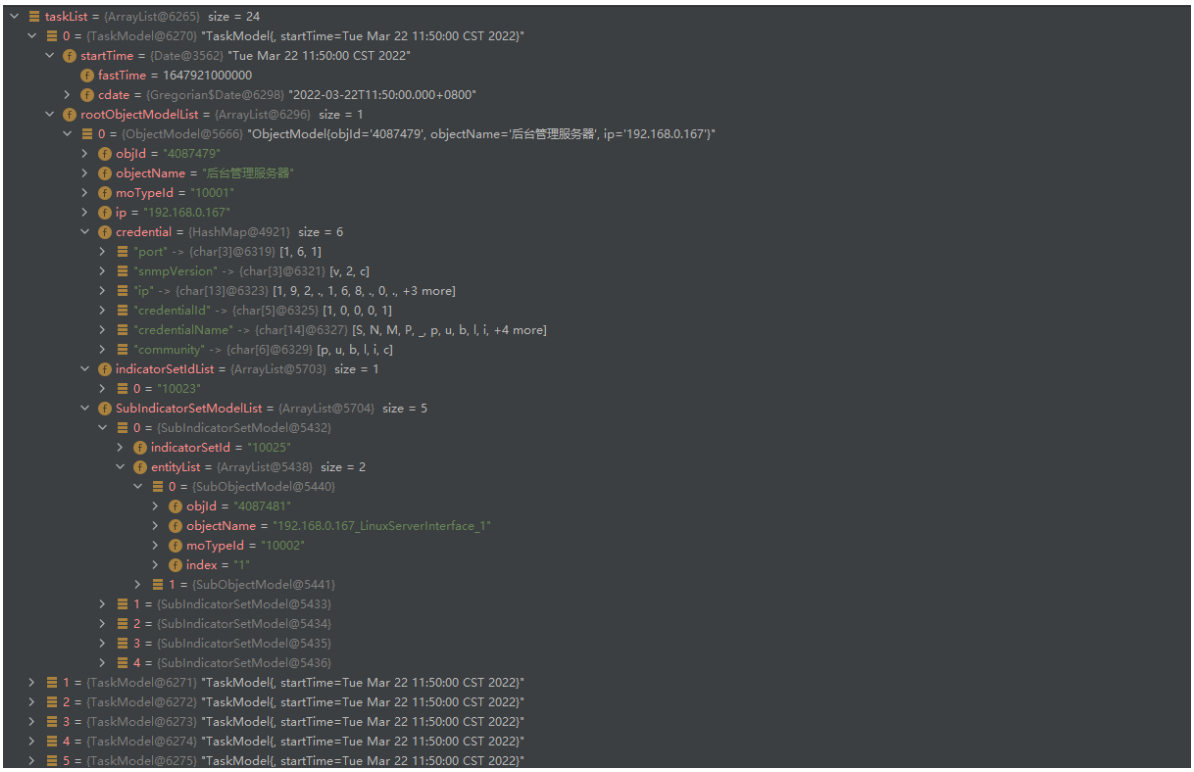
```
 IndicatorSetModel indicatorSetModel =
IndicatorsManager.indicatorSetModelMap.get(indicatorSetId);
 long period = Long.parseLong(indicatorSetModel.getDataPeriod());
 IndicatorSetModel indicatorSetModel =
IndicatorsManager.indicatorSetModelMap.get(indicatorSetId);
 long period = Long.parseLong(indicatorSetModel.getDataPeriod());
```

- 对子资源的指标集进行相同过滤
- 获得符合时间间隔的采集对象 objMap
- 遍历objMap 生成采集任务实体列表

```
List<TaskModel> taskList = new ArrayList<>();
for (Map.Entry<String, ObjectModel> entry : objectMap.entrySet())
    {
        TaskModel taskModel = new TaskModel(startTime);
        taskModel.setRootObjectModelList(new ArrayList<>());
        taskModel.getRootObjectModelList().add(entry.getValue());
        taskList.add(taskModel);

    }
```

- startTime 任务开始时间
- rootObjectModelList 被管对象信息列表




```
TaskQueue.getInstance().commit(taskList);
```

遍历采集对象，开启一个新的线程放入线程池中来进行采集任务

```
ListenableFuture<FutureModel> future =
    ThreadPoolManager.getInstance().submit(new TaskThread(rootObjectModel,
taskModel.getStartTime()));
```

任务线程：

```
public TaskThread(ObjectModel rootObjectModel, Date startTime)
{
    this.rootObjectModel = rootObjectModel;
    this.startTime = startTime;
}
```

传入参数：

- 采集对象
- 任务开始时间

线程调用**ProbeExecutor**协议采集实现实例 通过**executeTask（）**方法来进行采集任务

```
 ProbeExecutor t = null;
// 执行一个顶层对象的任务
try
{
    t = new ProbeExecutor();
    return t.executeTask(rootObjectModel, startTime);
}
```

executeTask（） 采集任务

1、连接

- 获取采集实现类 即**ICollector**接口的实现类 （snmp协议的实现类为**SnmpCollectorImpl**）

```
// 获取采集实现类
collect = (ICollector) ProbeContextHolder.getBean("collector");
```

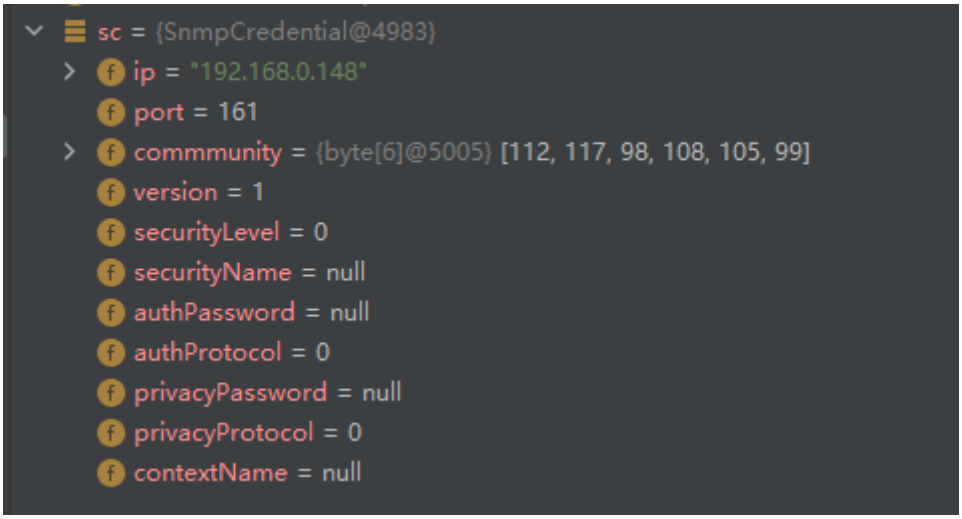
- 连接需要被采集的设备，返回连接结果

```
// 连接被管对象,传入凭证信息
collect.connect(rootObjectModel.getCredential());
```

connect：

（1）校验传入的采集凭证格式，如果有问题则抛出异常，没问题则返回一个采集凭证实体对象

```
SnmpCredential sc = this.checkSnmpCredential(credentials);
```



（2）传入当前时间和超期时间，新建一个Snmp协议客户端工具进行连接 （如果配置文件中没有超期时间则默认为3分钟）

```
snmpClient = new SnmpClient(new Date(), collectTimeOut);
```

利用客户端工具进行连接

```
isConnect = snmpClient.connect(sc);
```

2、连接成功后进行采集和计算

利用指标采集方法类 **CollectManager**进行采集 返回**ResultModel**

```
ResultModel resultModel = collectManager.collectAndCalculate(rootObjectModel,
collect, startTime, isConnect);
```

传入参数:

- rootObjectModel 被管对象
- collect 采集实现类
- startTime 开始时间
- isConnect 是否进行连接

collectAndCalculate:

（1）采集设备指标

- 遍历**indicatorSetIdList**指标集 获取采集结果模型

```
ResultObjectModel resultObjectModel =
    getResultObjectModel(indicatorSetId, rootObjectModel.getObjId(),
                         null, collect, isConnect, starTime, valuesMap, logMap);
```

getResultObjectModel

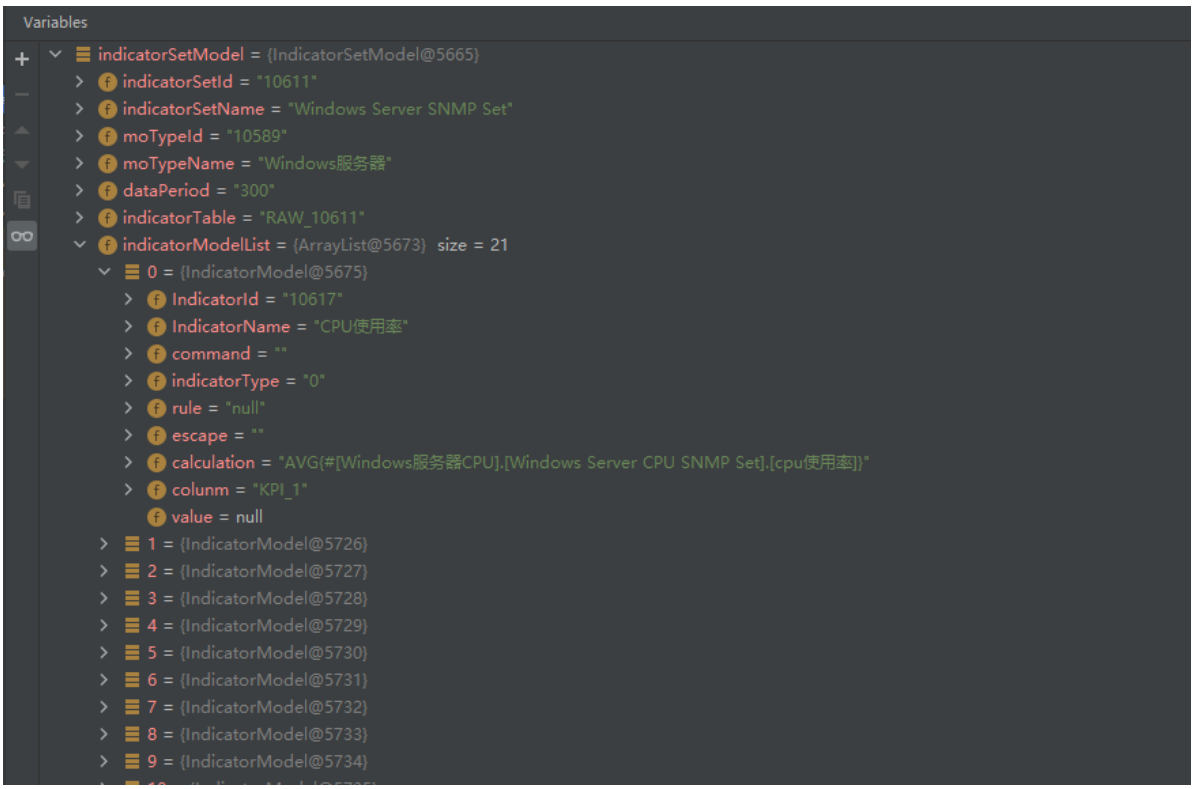
```
private ResultObjectModel getResultObjectModel(String indicatorSetId, String
objId, String index,
            ICollector collect, boolean isConnect, Date starTime, Map<String,
Map<String, Double>> valueCacheMap,
            Map<String, String> logMap)
```

传入参数:

- indicatorSetId 指标集ID
- rootObjectModel.getObjId() 模型ID
- index
- collect 采集实现对象
- isConnect 是否连接
- starTime 开始时间
- valueCacheMap 结果缓存集
- logMap 日志

1、根据指标集ID获取指标集对象

```
 IndicatorSetModel indicatorSetModel =
IndicatorsManager.indicatorSetModelMap.get(indicatorSetId);
```



2、遍历指标集内的指标对象，生成对应的KEY

```
String valueCacheKey = "[" + indicatorSetModel.getMoTypeName() + "]." + "["
        + indicatorSetModel.getIndicatorSetName() + "]." + "[" +
indicatorModel.getIndicatorName() + "]";
```

例如：[Linux服务器].[Linux Server SNMP Set].[文件系统使用率]

如果是普通指标 即type为1 则进行采集

```
collectResult = collect.collect(indicatorModel.getCommand(), index);
```

如果是KPI指标，则先存入计算公式